LEDGERLY

"Upload a statement. Understand your money."

A personal finance web application that transforms bank statement CSV files into categorized transactions, budgets, and visual dashboards — with no bank login required.

Document Type: Project Documentation & Technical Design
Tech Stack: Spring Boot 3 · React 18 · TypeScript · PostgreSQL 16
Version: 1.0
Date: October 2026

Table of Contents

1. Project Title & Overview
2. The Problem We're Solving
3. Reference Applications — 6 Working Apps with Identified Gaps
4. Gap Analysis — Where Ledgerly Fits In
5. GitHub Repository
6. Document Design — Description (Core Features)
7. Tech Stack & Justification
8. Data Flow Diagram (DFD)
9. System Architecture
10. Use Case Diagram
11. Entity-Relationship (ER) Diagram
12. The Import Pipeline — Deep Dive
13. API Design
14. Security Design
15. Development Roadmap
16. Why This is a Strong Portfolio Project

1. Project Title & Overview

Project Name: Ledgerly

One-liner: Ledgerly turns a messy bank statement CSV into a clean, categorized view of your money, with no bank login needed.

Most budgeting apps force you to hand over your bank credentials to a third-party aggregator like Plaid before you can even see your own spending. Ledgerly takes the opposite approach. You download a CSV from your bank's website — something every bank supports — and drop it into Ledgerly. The app parses the file, detects duplicates, applies your categorization rules, and shows you a full preview before anything gets saved. If something looks off, you fix it right there in the preview. When you're satisfied, you confirm, and the transactions are committed in a single atomic operation. If you later realize the whole import was wrong, one click undoes it entirely.

There's no account linking, no OAuth dance with your bank, no API tokens. Your financial data stays on your machine (self-hosted) or in your own account (hosted). That's the core promise: **privacy by design, clarity by default.**

> **Tagline Options:**
> • "Upload a statement. Understand your money."
> • "Your bank statements, made sense of."
> • "Clarity from a CSV."

2. The Problem We're Solving

Most people's relationship with their bank statements looks like this:

1. **The bank app shows raw data.** You get a list of transactions with cryptic descriptions like `POS 2847391 SWIGGY INSTA 01/10` and zero useful grouping. There's no sense of "where is my money actually going?"

2. **Built-in categories are useless.** Banks auto-tag "SWIGGY" as "Online Shopping" instead of "Food." You can't fix it. The categories are hardcoded and generic.
3. **Budgeting apps want your credentials.** Tools like Mint (now dead), YNAB, and Monarch Money push you to link your bank via Plaid or similar aggregators. That means handing your login to a third-party service — a non-starter for privacy-conscious users, especially in countries where open banking APIs don't exist or where bank agreements explicitly warn against credential sharing.
4. **CSV import is an afterthought everywhere.** The apps that do support CSV treat it as a second-class feature — rigid formats, no preview, terrible duplicate handling, and no undo. You upload a file, cross your fingers, and hope the parser doesn't choke on your bank's quirky column layout.

> **The Gap:** There is no clean, privacy-first tool that makes CSV-based statement management genuinely pleasant. Every existing solution either requires bank credentials, treats CSV as an afterthought, or demands sysadmin-level skills to set up. Ledgerly fills that gap.

3. Reference Applications — 6 Working Apps with Identified Gaps

We studied six existing applications that handle personal finance and CSV-based transaction management. For each one, we looked at what they do well, where they fall short, and what we can learn from their shortcomings. The goal isn't to copy any of them — it's to identify the gaps that, collectively, create the space Ledgerly is designed to fill.

3.1 Mint (by Intuit) → Now Credit Karma

Attribute	Details
What it did	Free budgeting app with automatic bank syncing, spending tracking, bill reminders, and credit score monitoring.
Status	Shut down March 23, 2024. Users forcibly migrated to Credit Karma.
Bank login required?	Yes — Plaid-based credential sharing was mandatory for core features.
CSV support	Export-only. Mint allowed CSV *downloads* of your history, but had no CSV *import* feature.
Pricing	Free (ad-supported — Intuit harvested financial data for targeted advertising).
Website	mint.intuit.com (redirects to Credit Karma)

Strengths: Massive user base (3.5M+ at peak), solid automatic categorization with years of ML training, beautiful trend charts and month-over-month comparisons, completely free to use.

Gaps & Weaknesses:

• **No CSV import at all.** If your bank wasn't supported by Plaid, you simply couldn't use Mint.

• **Privacy nightmare.** Intuit used financial data for targeted advertising and cross-selling TurboTax, insurance, and credit card products. "If it's free, you're the product" was the Reddit consensus.

• **Shutdown wiped user data.** Users who missed the narrow export window lost a decade of financial history permanently. Those who did export got raw CSVs with broken formatting that other apps struggled to ingest.

• **Credit Karma is not a replacement.** It focuses on credit scores and product recommendations, lacking budgeting, deep categorization, and long-term trend analysis.

What Ledgerly learns: A privacy-first, file-based approach means the user always owns their data. No company shutdown can take it away. And CSV import should be a *core feature*, not missing entirely.

3.2 YNAB (You Need A Budget)

Attribute	Details
What it does	Zero-based envelope budgeting app. Every dollar gets assigned a job.
Bank login required?	Optional (Plaid-based "Direct Import"), but heavily pushed.
CSV support	Yes — file import with column mapping (added after years of user complaints).
Pricing	$14.99/month or $109/year. No free tier.
Tech stack	Ruby on Rails backend, Ember.js frontend, PostgreSQL (via Crunchy Bridge).
Website	ynab.com

Strengths: Best-in-class budgeting methodology that genuinely changes how people think about money. Recently improved CSV import with in-app column mapping and payee name

cleansing. Excellent community and documentation. "YNAB Together" sharing for up to 6 people.

Gaps & Weaknesses:

• **CSV import is web/iPad only.** No mobile file import — a pain point for users without regular desktop access.

• **No import preview.** You map columns, hit import, and pray. If something's wrong, you manually delete transactions row-by-row or undo the entire import (which also wipes any edits you made to those rows).

• **Duplicate handling is fragile.** Mixing Plaid feeds with CSV uploads frequently creates double entries. Deduplication relies on fuzzy date/amount matching that breaks when dates shift by even one day.

• **Rigid CSV format expectations.** Separate debit/credit columns, European number formatting (`1.234,56`), or multi-header rows cause import failures.

• **Expensive.** $14.99/month is steep for someone who just wants to see where their money went.

What Ledgerly learns: A genuine *preview before save* flow and robust duplicate detection are non-negotiable. YNAB's "import and hope" approach is a known frustration — we can do better.

3.3 Firefly III (Open-Source, Self-Hosted)

Attribute	Details
What it does	Self-hosted personal finance manager with double-entry bookkeeping, budgets, categories, piggy banks, and recurring transactions.
Bank login required?	No (CSV/OFX import). Optional Nordigen/GoCardless for EU open banking.
CSV support	Yes — via a *separate companion app* ("Firefly III Data Importer").
Pricing	Free and open-source (AGPL-3.0).
Tech stack	Laravel (PHP), MySQL/PostgreSQL, Blade templates.

Attribute	Details
GitHub	github.com/firefly-iii/firefly-iii

Strengths: Fully open-source with complete data ownership. Powerful rule engine for auto-categorization. Double-entry bookkeeping for accountants. Active development community. Multi-currency support. Full REST API.

Gaps & Weaknesses:

- **Import requires a separate application.** The Data Importer communicates with Firefly III via OAuth + API. Setting up two Docker containers with correct networking trips up even experienced users.
- **Performance craters on large files.** Sequential row processing with rules enabled means a 2,000-row CSV can take *hours*. 504 Gateway Timeouts are routine during imports.
- **No import preview.** Transactions go directly into the database. Miscategorized or wrongly mapped data is discovered after the fact.
- **Locale/format issues are rampant.** European number formats, date inconsistencies, and mid-file header rows cause silent failures.
- **No batch undo for imports.** Once imported, transactions are individual records with no "undo this batch" concept.
- **High barrier to entry.** Requires Docker, reverse proxy, OAuth, and Linux sysadmin knowledge.

What Ledgerly learns: An import that *just works* — no Docker compose, no OAuth tokens, no separate apps — is a massive differentiator. Preview and batch undo are features Firefly III users actively request on GitHub Issues.

3.4 Monarch Money

Attribute	Details
What it does	All-in-one financial dashboard: budgeting, investment tracking, net worth, cash flow, and household collaboration. Positioned as the "Mint replacement."

Attribute	Details
Bank login required?	Yes — Plaid/MX/Finicity-based syncing is the primary workflow.
CSV support	Yes — file import exists but is clearly a secondary feature.
Pricing	$14.99/month or $99.99/year. No free tier.
Tech stack	Django (Python), React, GraphQL, PostgreSQL, Redis, AWS.
Website	monarchmoney.com

Strengths: Gorgeous, polished UI with investment tracking, net worth timelines, and crypto support. Built dedicated Mint data importers during the 2024 migration. Multi-user household sharing. Overlap management in CSV imports.

Gaps & Weaknesses:

• **CSV import is barebones.** Minimal mapping options, limited error feedback — clearly optimized for Plaid-synced accounts.

• **Privacy concerns.** Aggregates bank accounts, credit cards, investments, crypto, and real estate — creating the largest data footprint of any app. All stored on their cloud.

• **No rule-based categorization.** Uses ML-based auto-categorization that users report has *degraded over time*. No deterministic rules like "if description contains X, categorize as Y."

• **No import undo.** Imported CSV transactions merge into the account. Rolling back means manual deletion.

• **Paid-only.** ~$100/year with no free tier is a barrier for students.

What Ledgerly learns: ML-based categorization is a black box that can regress. User-defined, deterministic rules give predictability and control. Also, a free option matters.

3.5 Wallet by BudgetBakers

Attribute	Details
What it does	Multi-platform personal finance app with manual entry, bank syncing (via Salt Edge), budgets, reports, CSV/XLS import.
Bank login required?	Optional (Salt Edge for supported banks).
CSV support	Yes — with column mapping, supports CSV, XLS, XLSX, OFX.
Pricing	Free tier with limits. Premium ~$5.49/month or $34.99/year. Lifetime license option.
Tech stack	Go & Scala backend, React web frontend, native mobile (Kotlin, Swift).
Website	budgetbakers.com

Strengths: Multi-platform with syncing, supports multiple file formats, affordable premium tier with lifetime option, 15,000+ bank institution coverage, column mapping UI.

Gaps & Weaknesses:

- **CSV import disabled for bank-linked accounts.** If an account is connected via Salt Edge, CSV import is completely blocked for it. You can't mix methods.
- **1,000-row limit.** Larger files freeze, crash, or silently fail.
- **Brutally strict parser.** No thousands separators (`1,000.00` breaks it), amounts must be formatted left-to-right (`100.00-` fails), single currency per file only.
- **No preview before save.** Mistakes are discovered after import.
- **No batch undo.** Imported transactions scatter. Rolling back means manual deletion.

What Ledgerly learns: Format flexibility is crucial. A parser that chokes on `1,000.00` or a trailing negative sign is unacceptable. Preview + batch undo should be standard.

3.6 Actual Budget (Open-Source)

Attribute	Details
What it does	Local-first, open-source personal finance app with envelope budgeting, CSV/OFX import, and optional bank syncing via SimpleFIN or GoCardless.
Bank login required?	No — CSV import is a core workflow. Optional bank syncing available.
CSV support	Yes — built-in column mapping with format memory.
Pricing	Free and open-source (MIT license).
Tech stack	Node.js, TypeScript, React, SQLite (local-first architecture).
GitHub	github.com/actualbudget/actual

Strengths: Beautiful, fast UI. Local-first data architecture (works offline, syncs when connected). Genuinely treats CSV import as a first-class feature. Active open-source community with frequent releases. Envelope budgeting similar to YNAB but free.

Gaps & Weaknesses:

• **No preview of categorized results before save.** You can map columns and see raw data, but there's no rule-engine preview showing which categories will be assigned before committing.

• **Duplicate detection is basic.** Relies on date + amount + payee matching, which breaks with slightly different payee strings across statements.

• **No categorization rule engine.** Payee renaming rules exist, but there's no condition-based system for auto-categorizing (e.g., "contains SWIGGY → Food"). Categorization is learned from manual assignments.

• **No import batch undo.** Imported transactions merge into the ledger. No concept of "undo this upload."

• **Requires self-hosting for sync.** Local-only works fine, but multi-device sync requires running a server.

What Ledgerly learns: Actual Budget proves there's strong demand for open-source, CSV-first finance tools. But even the best open-source option lacks a rule engine, categorized preview, and batch undo — exactly the features Ledgerly builds around.

4. Gap Analysis — Where Ledgerly Fits In

After studying all six competitors, a clear pattern emerges. Here's the consolidated gap map showing how each tool handles the features that matter most for CSV-based personal finance:

Capability	Mint	YNAB	Firefly III	Monarch	Wallet	Actual	Ledgerly
CSV import as core feature	✗ None	⚠ Secondary	⚠ Separate app	⚠ Secondary	⚠ Limited	✓ Good	✓ Core
No bank credentials required	✗	⚠ Optional	✓	✗	⚠ Optional	✓	✓ Never
Preview before saving	✗	✗	✗	✗	✗	✗	✓ Yes
Batch undo (rollback import)	✗	⚠ Partial	✗	✗	✗	✗	✓ Yes
Robust duplicate detection	✗	⚠ Fragile	⚠ Slow	⚠ Basic	⚠ Basic	⚠ Basic	✓ Hash
User-defined categorization rules	✗	⚠ Limited	✓	✗ ML only	⚠ Weak	✗	✓ Full
Handles messy bank formats	✗	✗	⚠ Manual	✗	✗	⚠ Basic	✓ Profiles
Free / self-hostable	✗ Dead	✗ $15/mo	✓	✗ $10/mo	⚠ Freemium	✓	✓ Free

> **Ledgerly's Unique Position:** Ledgerly is the **only tool** that combines all seven: CSV-first design, zero credential sharing, preview-before-save, one-click batch undo, hash-based deduplication, a full deterministic rule engine, and saved bank profiles for messy formats — all while being free and open-source.

5. GitHub Repository

Attribute	Details
Repository Name	`ledgerly`
Repository Link	https://github.com/aarav/ledgerly
Visibility	Public
License	MIT
Primary Language	Java (backend) + TypeScript (frontend)

Suggested Repository Structure

```
ledgerly/
├── README.md
├── docs/
│   └── PROJECT_DOCUMENTATION.pdf
├── backend/                          # Spring Boot application
│   ├── src/main/java/com/ledgerly/
│   │   ├── config/                   # Security, CORS, JWT config
│   │   ├── controller/               # REST API controllers
│   │   ├── service/                  # Business logic (import, rules, budget)
│   │   ├── repository/               # Spring Data JPA repositories
│   │   ├── model/                    # JPA entity classes
│   │   ├── dto/                      # Request/Response DTOs
│   │   ├── engine/                   # Rule engine implementation
│   │   └── scheduler/                # Scheduled jobs (budget alerts)
│   ├── src/main/resources/
│   │   ├── application.yml
│   │   └── db/migration/             # Flyway SQL migrations
│   └── pom.xml
├── frontend/                         # React + TypeScript application
│   ├── src/
│   │   ├── components/               # Reusable UI components
│   │   ├── pages/                    # Page-level components
│   │   ├── hooks/                    # Custom React hooks
│   │   ├── api/                      # API client (Axios/fetch wrappers)
│   │   └── utils/                    # CSV parsing, formatting helpers
│   ├── package.json
│   └── vite.config.ts
├── docker-compose.yml                # Full stack: backend + frontend + PostgreSQL
└── .github/workflows/ci.yml          # GitHub Actions CI/CD pipeline
```

6. Document Design — Description (Core Features)

This section describes what Ledgerly does in detail — the features a user interacts with, and the engineering that powers them behind the scenes.

6.1 Smart CSV Import

Ledgerly's import system is designed to handle the messy reality of bank CSV files. Every bank formats their exports differently — different column orders, different date formats, different ways of representing debits and credits. Instead of requiring users to manually reformat their files before uploading, Ledgerly adapts to the file.

- **Upload any bank's CSV file** — no reformatting needed.
- **Column mapping UI** — select which column is date, amount, description, etc.
- **Save as a Bank Profile** — map once for your bank, every future upload auto-maps.
- **Edge case handling** — separate debit/credit columns, locale-specific number formats (`1.234,56`), varying date formats (`DD/MM/YYYY` vs `MM-DD-YYYY`), mid-file header rows.

6.2 Duplicate Protection

Re-uploading overlapping statements should be safe, not anxiety-inducing. Ledgerly generates a SHA-256 fingerprint for every transaction based on its date, amount, description, and account. On import, these fingerprints are checked against existing records. Duplicates aren't silently skipped or silently imported — they're **flagged in the preview** so the user decides what happens.

6.3 Rule-Based Categorization

Instead of relying on a black-box ML model that might categorize "SWIGGY" as "Online Shopping" next month, Ledgerly uses deterministic rules that the user controls:

- `IF description CONTAINS "SWIGGY" THEN category = "Food"`
- Supports: `contains`, `starts with`, `ends with`, `regex`, `exact match`, `amount greater/less than`
- Rules have priority ordering — first match wins.
- During preview, you see *which rule* matched each transaction, making the system transparent and debuggable.

6.4 Preview Before Save

This is Ledgerly's most important UX feature. After upload + mapping + deduplication + categorization, you see a full preview table showing every transaction with its matched category, matched rule, and duplicate status. You can:

- Edit any field inline — change a category, fix a description, adjust an amount
- Remove rows you don't want to import
- Create a new categorization rule on the spot
- Only when you click **"Confirm Import"** are transactions saved to the database

6.5 Import Undo

Every confirmed import is tagged with a batch ID. From the import history page, you can undo an entire import with one click — deleting all transactions from that batch. Budget and dashboard numbers recalculate automatically. This is possible because every transaction stores its `import_batch_id`.

6.6 Budgets & Alerts

Set monthly spending limits per category (e.g., "Food: ₹8,000/month"). The dashboard shows progress bars with current spending versus budget. Warnings appear at 80%

utilization. At 100%, an overspend alert fires. Alerts can optionally trigger email notifications via a scheduled backend job.

6.7 Dashboard & Analytics

- **Spending by category** — pie/donut chart for the current month
- **Income vs. Expense** — bar chart showing monthly net cash flow
- **Trends over time** — line chart showing category-wise spending across months
- **Top merchants** — ranked list of where you spend the most
- **Filter by date range, account, or category**

7. Tech Stack & Justification

Every technology choice was made for a specific reason. Here's the full stack and the thinking behind each pick:

Layer	Technology	Why This Choice
Backend	Spring Boot 3.x (Java 21)	Enterprise-grade framework with strong typing for financial calculations, `@Transactional` for data integrity, massive ecosystem of libraries and tooling.
Frontend	React 18 + TypeScript + Vite	Component-driven UI ideal for data tables, modals, and interactive dashboards. TypeScript catches type errors at compile time — critical when handling financial data.
Database	PostgreSQL 16	ACID-compliant with `NUMERIC` type for exact money arithmetic (no floating-point rounding errors). Powerful aggregation functions, JSONB support, full-text search.
ORM	Spring Data JPA + Hibernate	Eliminates boilerplate with derived query methods. Handles schema management alongside Flyway.
Migrations	Flyway	Version-controlled SQL migrations. No schema "drift" between dev, staging, and production.
Auth	Spring Security + JWT	Stateless authentication — no server-side session storage. Tokens expire and refresh cleanly.
Charting	Recharts	React-native charting library with clean API. Handles pie, bar, and line charts out of the box.
UI Components	Shadcn/UI + Tailwind CSS	Modern, accessible component library. Tailwind provides pixel-level layout control without custom CSS files.

Layer	Technology	Why This Choice
State Mgmt	TanStack React Query	Server state management with built-in caching, loading/error states, and automatic refetching of stale data.
CSV Parsing	Apache Commons CSV (backend) + PapaParse (frontend)	Commons CSV handles locale-aware parsing and large files server-side. PapaParse gives instant client-side preview without uploading.
Build/Deploy	Docker + Docker Compose	One command spins up the entire stack. Consistent environments everywhere.
CI/CD	GitHub Actions	Automated testing, linting, and Docker image builds on every push.
Scheduling	Spring `@Scheduled`	Budget alert emails and periodic cleanup. No external scheduler needed.
Email	Spring Mail + Thymeleaf	Budget overspend notifications with clean HTML email templates.

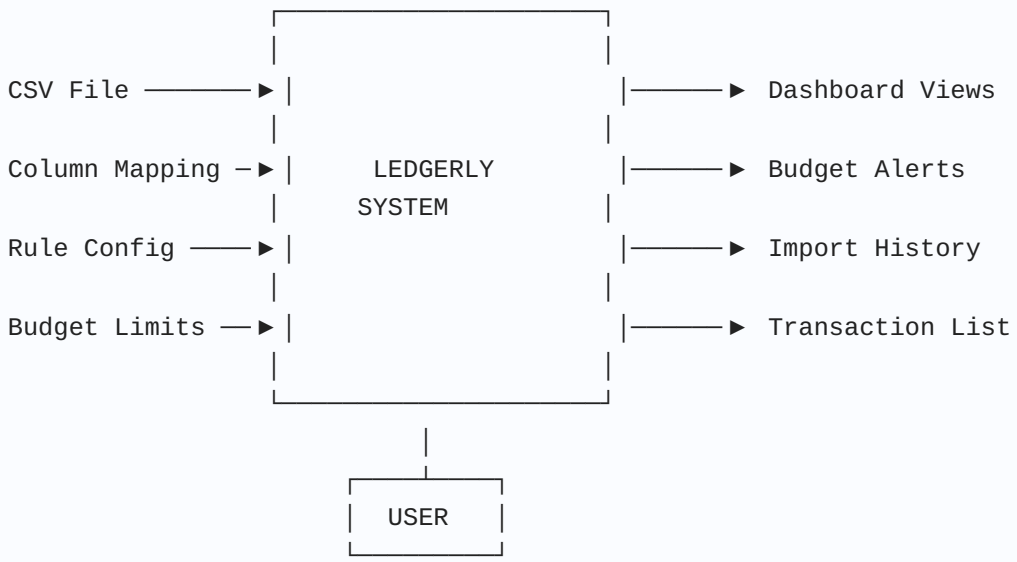
> **Why a Modular Monolith (Not Microservices)?**
> Ledgerly is a single-user or small household app. There's no need to scale the CSV parser independently of the dashboard API. A monolith is simpler to deploy, debug, and reason about — one JAR, one database, one log stream. Spring Boot's package structure provides clean module boundaries without the overhead of service mesh, inter-service auth, and distributed tracing. If it ever needs to scale, the modular structure makes extraction into services straightforward.

8. Data Flow Diagram (DFD)

The Data Flow Diagram shows how data moves through the Ledgerly system — from the moment a user selects a CSV file to the moment dashboard charts render on screen. This is a Level-1 DFD that captures the major processes and data stores.

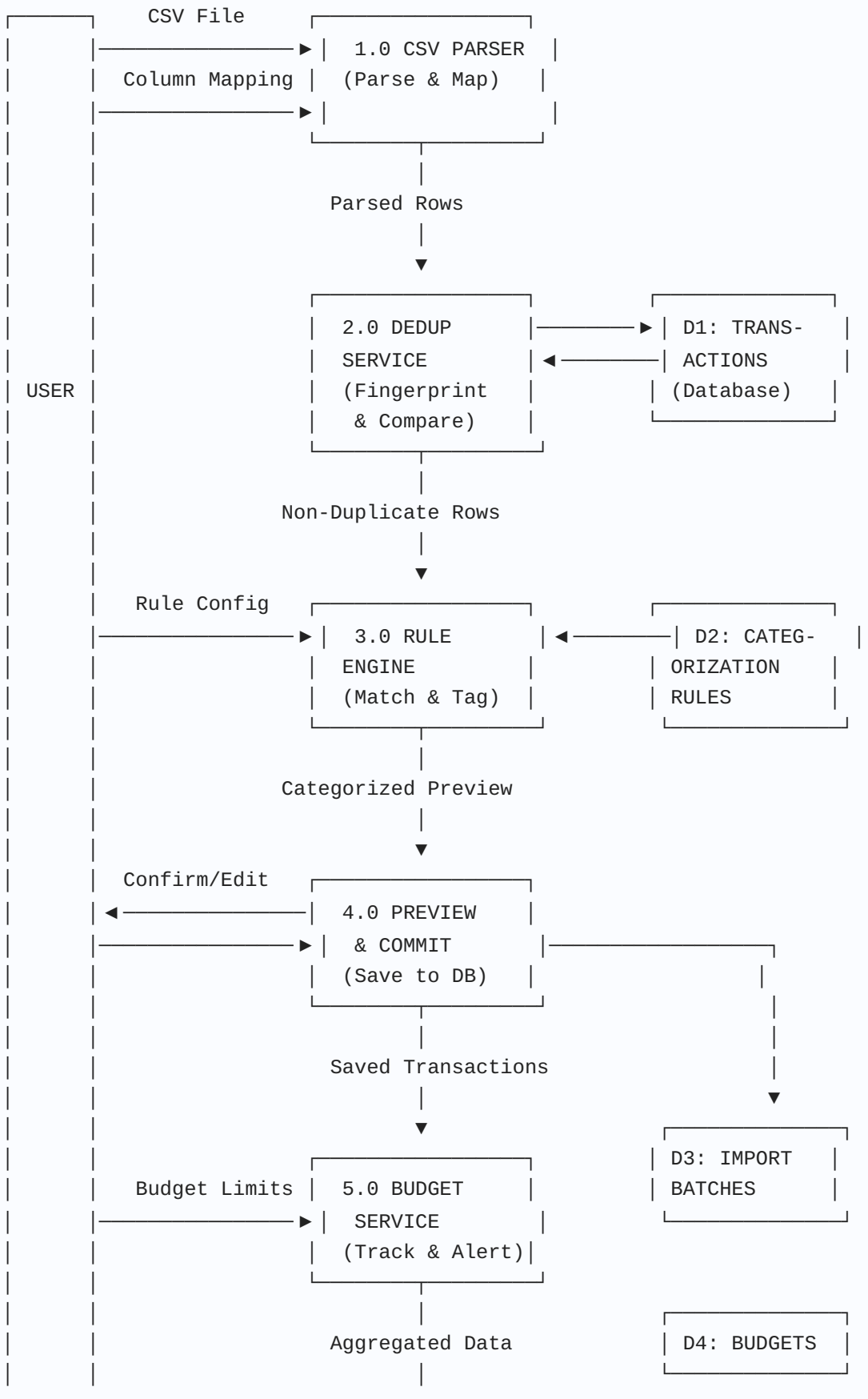
8.1 Level-0 (Context Diagram)

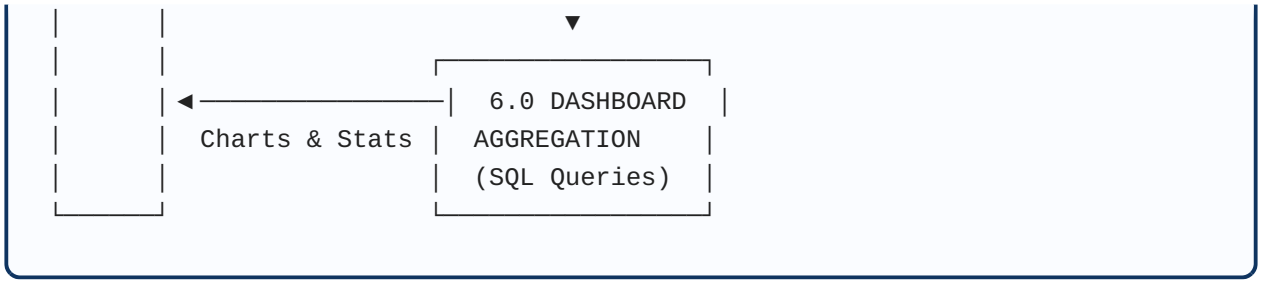
Level-0 DFD — Context Diagram



8.2 Level-1 (Detailed Data Flow)

Level-1 DFD — Internal Data Flow





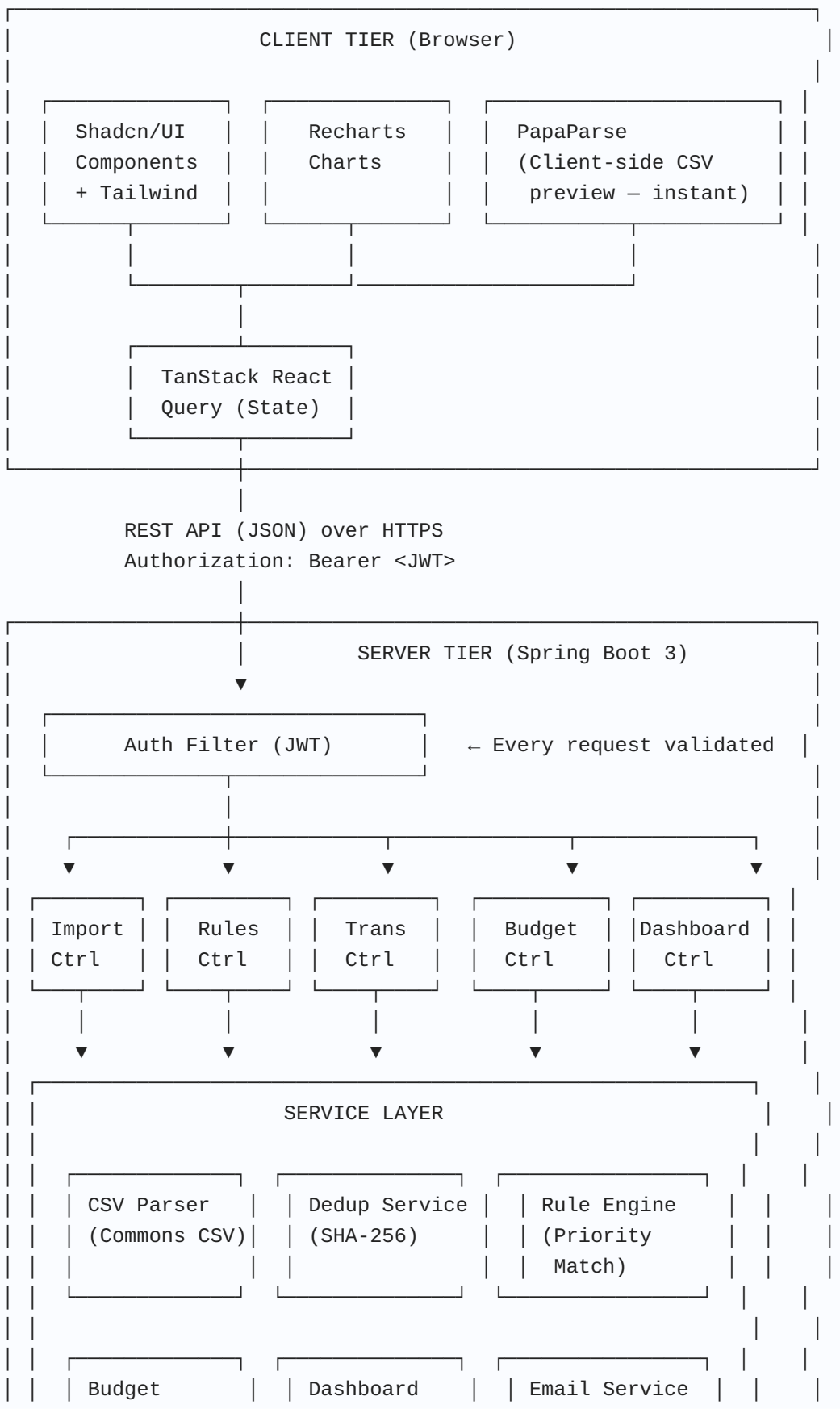
Data Flow Description

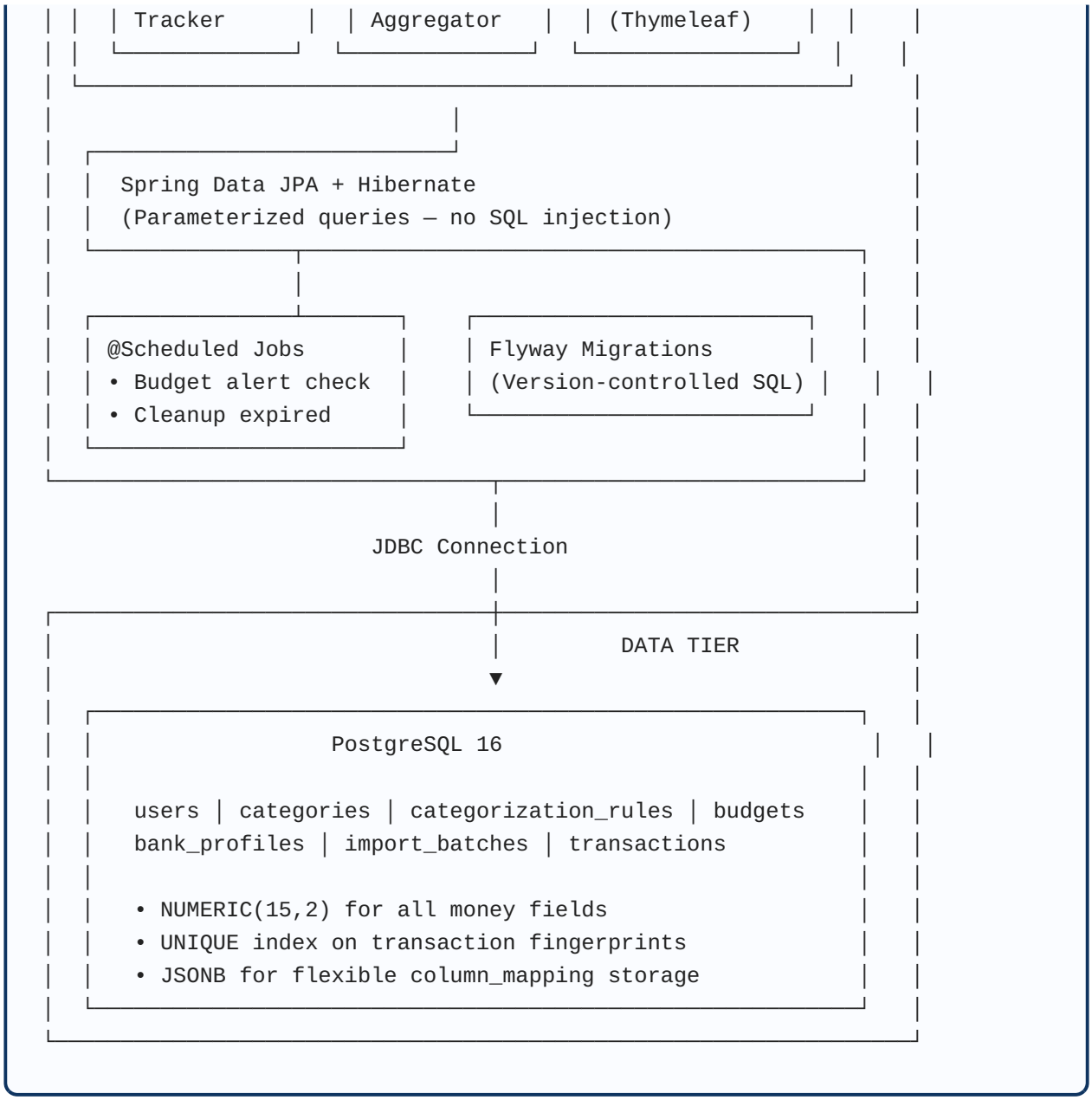
Flow	From	To	Data
1	User	CSV Parser	Raw CSV file + column mapping configuration
2	CSV Parser	Dedup Service	List of parsed, normalized transaction rows
3	Dedup Service	Transactions DB	SHA-256 fingerprints for comparison lookup
4	Dedup Service	Rule Engine	Non-duplicate rows needing categorization
5	Rule Engine	Rules DB	User's categorization rules (priority-ordered)
6	Rule Engine	Preview & Commit	Fully categorized preview with matched rules
7	Preview & Commit	User	Preview table for review and inline editing
8	User	Preview & Commit	Confirmed, edited transaction list
9	Preview & Commit	Transactions DB	Finalized transactions with batch_id
10	Budget Service	Dashboard	Aggregated spending vs. budget limits
11	Dashboard	User	Charts, trends, summaries, alerts

9. System Architecture

Ledgerly follows a clean three-tier architecture: a React single-page application communicates with a Spring Boot REST API, which in turn reads from and writes to a PostgreSQL database. A scheduled job layer handles background tasks like budget alert emails.

System Architecture Diagram

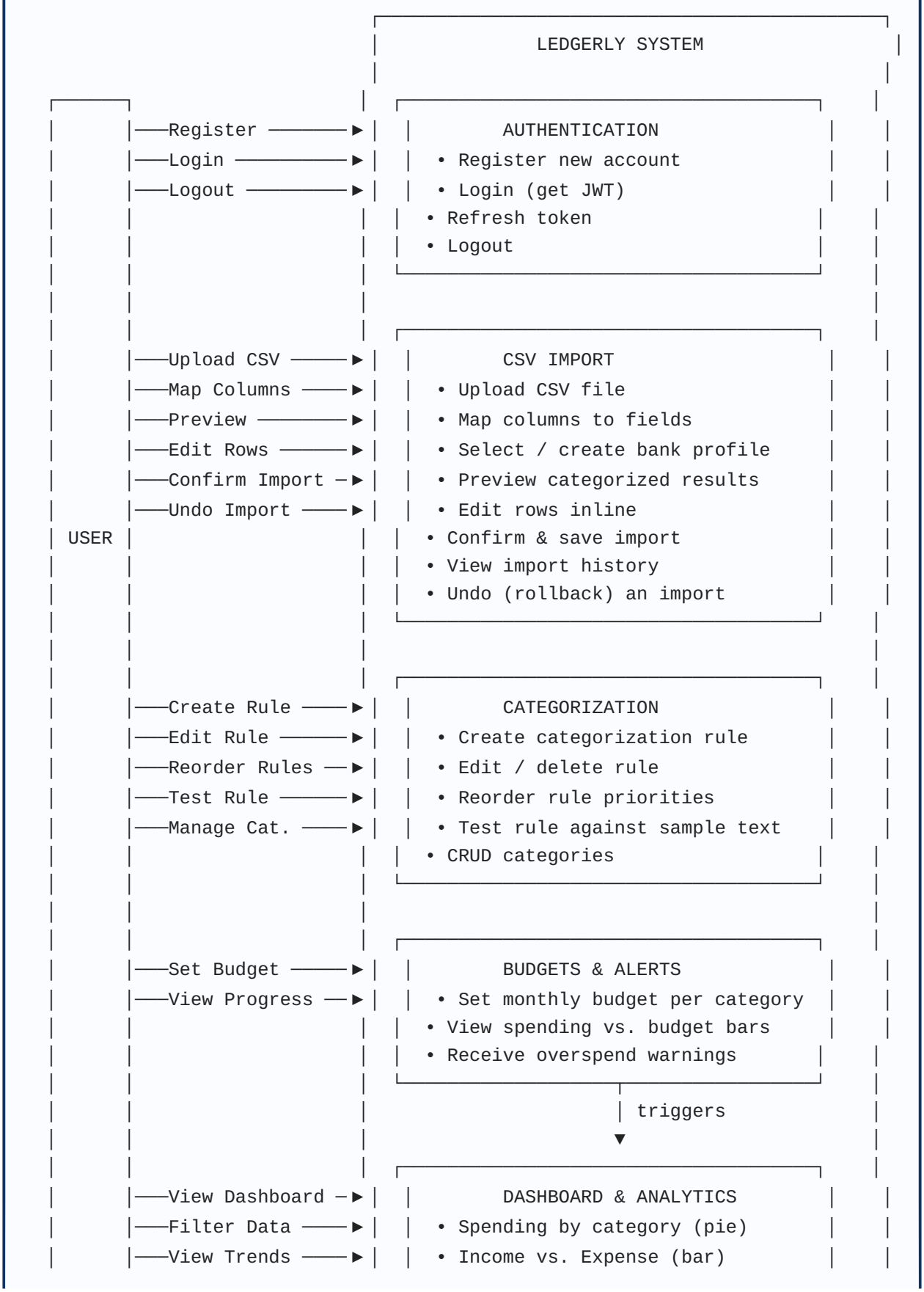


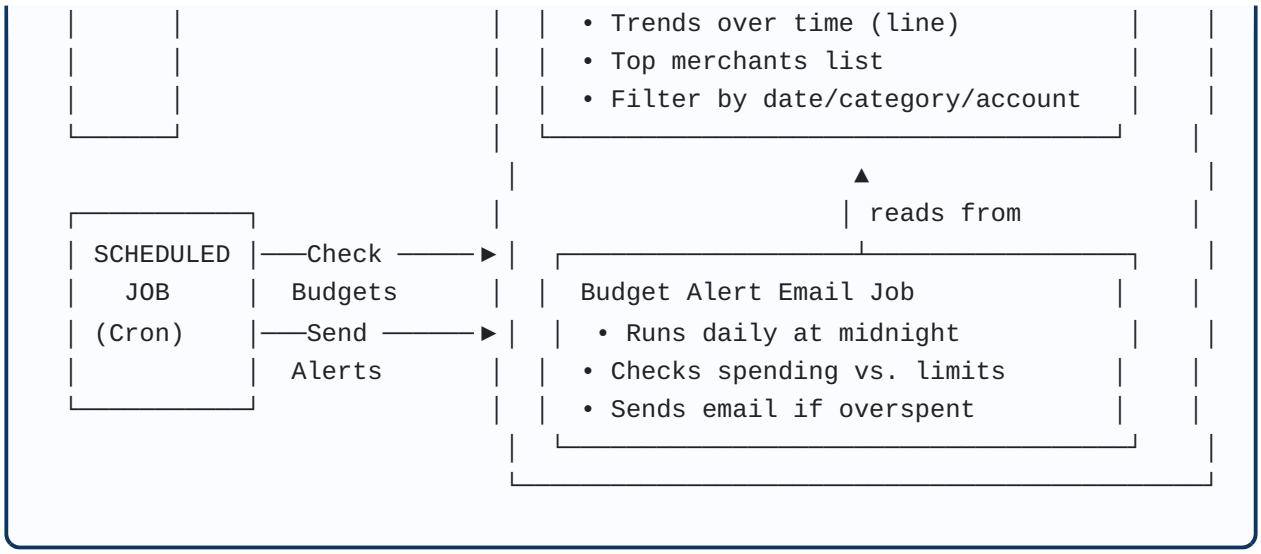

Tracker
Aggregator
(Thymeleaf)
Spring Data JPA + Hibernate
(Parameterized queries — no SQL injection)
@Scheduled Jobs
• Budget alert check
• Cleanup expired
Flyway Migrations
(Version-controlled SQL)
JDBC Connection
DATA TIER
PostgreSQL 16
users | categories | categorization_rules | budgets
bank_profiles | import_batches | transactions
• NUMERIC(15,2) for all money fields
• UNIQUE index on transaction fingerprints
• JSONB for flexible column_mapping storage

10. Use Case Diagram

The Use Case Diagram shows every action a user can perform in Ledgerly, organized by functional area. The system boundary includes all backend processes; the scheduled job actor represents automated background tasks.

Use Case Diagram — Ledgerly





Use Case Summary Table

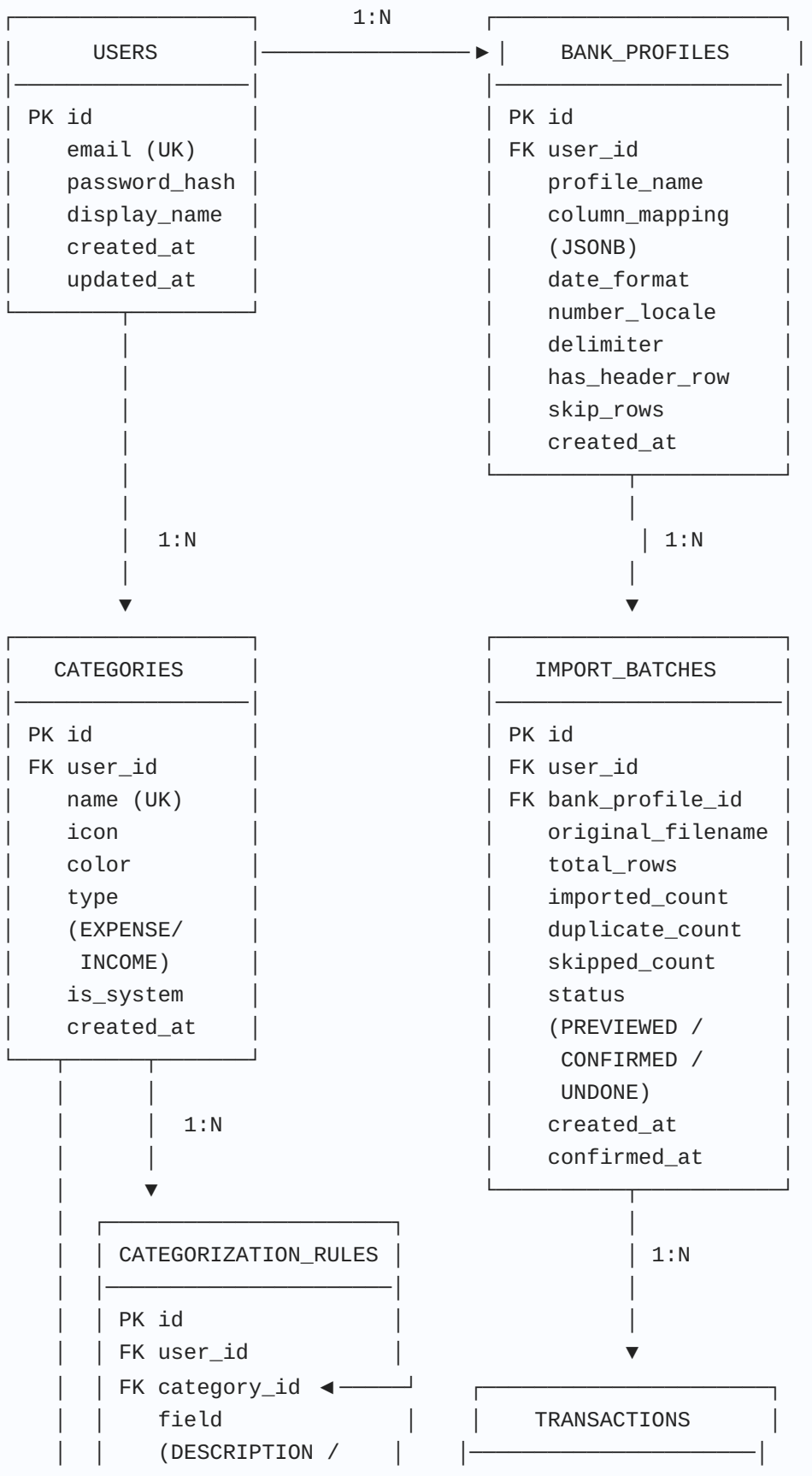
#	Use Case	Actor	Description
UC-01	Register	User	Create a new account with email and password
UC-02	Login	User	Authenticate and receive JWT access + refresh tokens
UC-03	Upload CSV	User	Select and upload a bank statement CSV file
UC-04	Map Columns	User	Map CSV columns to Ledgerly fields (or select saved profile)
UC-05	Preview Import	User	View categorized, deduplicated transactions before saving
UC-06	Confirm Import	User	Commit previewed transactions to database
UC-07	Undo Import	User	Rollback all transactions from a specific import batch
UC-08	Manage Rules	User	Create, edit, reorder, test categorization rules
UC-09	Manage Categories	User	Create, edit, delete spending categories
UC-10	Set Budget	User	Define monthly spending limit for a category
UC-11	View Dashboard	User	See spending charts, trends, and summaries

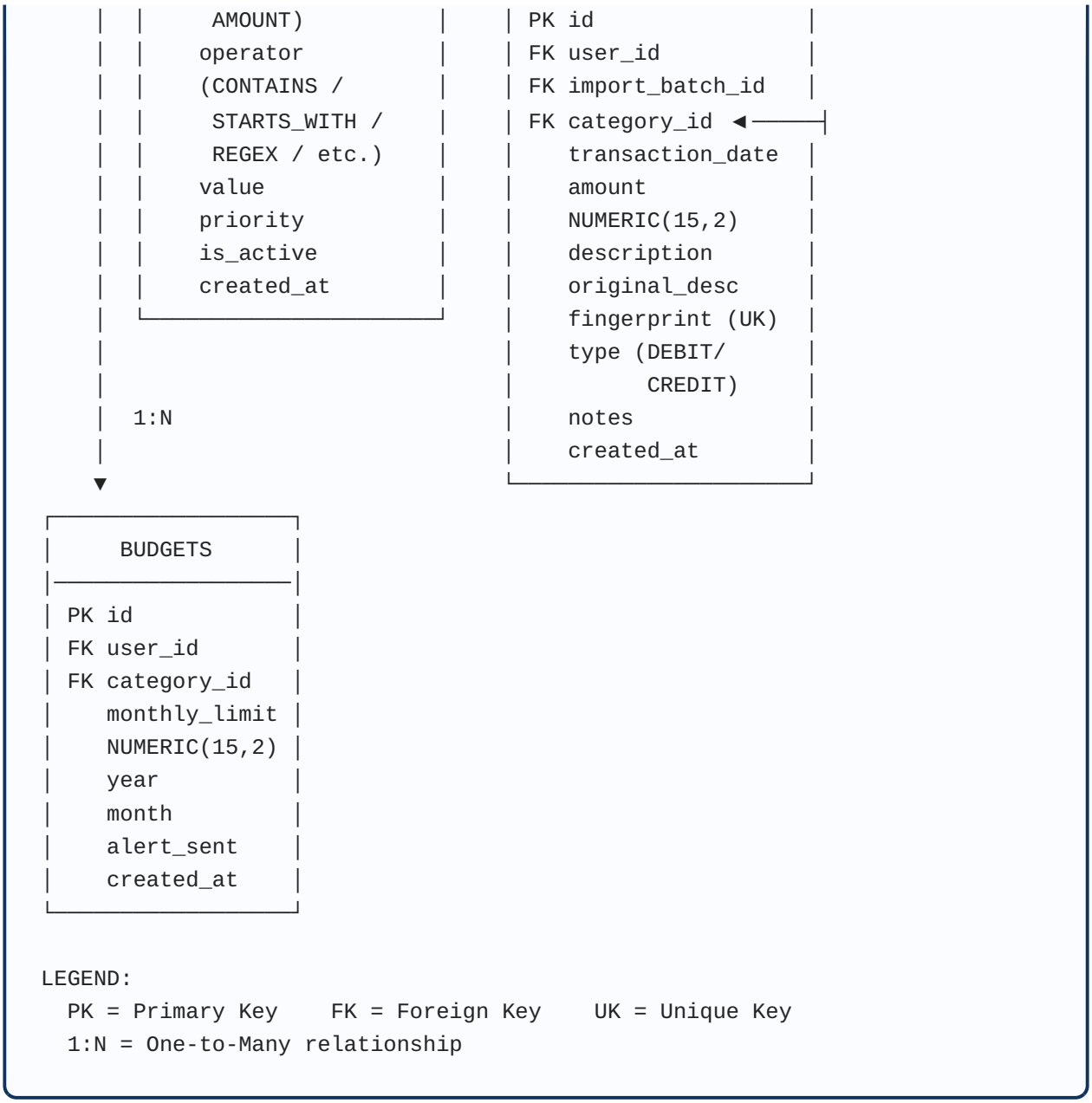
#	Use Case	Actor	Description
UC-12	Send Budget Alert	Scheduler	Auto-check budgets and email overspend notifications

11. Entity-Relationship (ER) Diagram

The database schema is built around seven core entities. The relationships between them enable every feature — from deduplication (fingerprint on transactions) to batch undo (import_batch_id on every transaction) to budget tracking (category_id linking budgets to transactions).

ER Diagram — Ledgerly Database Schema





Key Design Decisions

Decision	Rationale
`NUMERIC(15,2)` for money	Never `FLOAT` or `DOUBLE`. Floating-point arithmetic causes rounding errors in financial calculations. PostgreSQL's `NUMERIC` type provides exact decimal arithmetic.
`fingerprint` as unique key	`SHA-256(date + amount + description + user_id)` guarantees re-uploading the same statement never creates duplicates. Indexed for O(1) lookups via batch `IN` queries.

Decision	Rationale
`import_batch_id` on transactions	Enables batch undo: `DELETE FROM transactions WHERE import_batch_id = ?`. Clean, fast, atomic rollback of any import.
`original_description` preserved	When users edit descriptions, the raw bank text is kept separately. Ensures fingerprints remain valid and the original data is always recoverable.
`column_mapping` as JSONB	Bank profiles store mapping configs as flexible JSON objects: `{"date": 0, "description": 3, "amount": 4}`. Handles any CSV layout without schema changes.
Rule priority ordering	Rules evaluated in ascending `priority` order. First match wins. Gives users deterministic, predictable, debuggable categorization.

12. The Import Pipeline — Deep Dive

This is the most technically interesting part of Ledgerly and the strongest talking point in an interview. The pipeline has seven stages, each solving a specific problem.

1 Client-Side Preview (Instant)

The user selects a file. PapaParse reads the first 5 rows *entirely in the browser* — no upload happens yet. The UI instantly shows column headers and sample data, letting the user map columns or select a saved Bank Profile.

Why client-side first? Instant feedback. If they picked the wrong file, they know in under a second. The file never leaves the browser until the user is ready.

2 Server-Side Parsing

The file is uploaded via `POST /api/imports/preview` as a multipart request. Apache Commons CSV parses the full file using the Bank Profile settings (delimiter, date format, number locale, rows to skip). Output: a `List<RawTransaction>`.

Edge cases handled: Separate debit/credit columns merged into signed amounts. European numbers (`1.234,56`) normalized. Multiple date formats detected. Mid-file header rows skipped.

3 Fingerprinting & Deduplication

For each parsed row, a SHA-256 fingerprint is computed from `date + amount + description + userId`. All fingerprints are batch-checked against the database in a single `SELECT ... WHERE fingerprint IN (?)` query. Matches are flagged as `DUPLICATE` in the preview.

Why SHA-256? Fast, deterministic, collision-resistant. Stored as `VARCHAR(64)` with a unique index — making lookups effectively O(1).

4 Rule Engine

For each non-duplicate row, the user's categorization rules are loaded in priority order. Each rule is evaluated sequentially — first match wins, assigns the category, and records which rule matched. If no rule matches, the row is marked "Uncategorized."

Supported operators: `CONTAINS` (case-insensitive), `STARTS_WITH`, `ENDS_WITH`, `EXACT`, `REGEX` (Java Pattern), `AMOUNT_GT`, `AMOUNT_LT`.

5 Preview Response

The API returns a `List<PreviewTransaction>` containing: date, description, amount, type, assignedCategory, matchedRuleName, isDuplicate, and originalRowNumber. The frontend renders this as a rich, interactive data table.

6 User Review & Edit

The user reviews the preview table and can: change any category, edit descriptions or amounts, remove unwanted rows, or create a new rule inline ("Always categorize transactions containing X as Y"). Nothing has been saved yet.

7 Confirm & Save

`POST /api/imports/confirm` triggers a `@Transactional` method that creates an ImportBatch record, bulk-inserts all confirmed transactions with the batch_id, and updates budget progress via aggregate queries. Returns an ImportSummary with counts.

Atomicity: The entire confirm is a single database transaction. If anything fails, nothing is saved. The user can retry safely.

13. API Design

Authentication

Method	Endpoint	Description
`POST`	`/api/auth/register`	Create a new account
`POST`	`/api/auth/login`	Get JWT access + refresh tokens
`POST`	`/api/auth/refresh`	Refresh an expired access token

Bank Profiles

Method	Endpoint	Description
`GET`	`/api/bank-profiles`	List all saved bank profiles
`POST`	`/api/bank-profiles`	Create a new bank profile
`PUT`	`/api/bank-profiles/{id}`	Update a bank profile
`DELETE`	`/api/bank-profiles/{id}`	Delete a bank profile

CSV Import

Method	Endpoint	Description
`POST`	`/api/imports/preview`	Upload CSV + mapping → get preview (no save)
`POST`	`/api/imports/confirm`	Confirm previewed import → save to DB
`GET`	`/api/imports`	List past imports with stats
`DELETE`	`/api/imports/{batchId}`	Undo an import (delete batch transactions)

Transactions

Method	Endpoint	Description
`GET`	`/api/transactions`	List transactions (paginated, filterable)
`PUT`	`/api/transactions/{id}`	Update a transaction (category, notes)
`DELETE`	`/api/transactions/{id}`	Delete a single transaction

Categories & Rules

Method	Endpoint	Description
`GET`	`/api/categories`	List all categories
`POST`	`/api/categories`	Create a new category
`PUT/DELETE`	`/api/categories/{id}`	Update or delete a category
`GET`	`/api/rules`	List all rules (priority-ordered)
`POST`	`/api/rules`	Create a new rule
`PUT/DELETE`	`/api/rules/{id}`	Update or delete a rule
`POST`	`/api/rules/reorder`	Reorder rule priorities
`POST`	`/api/rules/test`	Test rules against sample text (dry run)

Budgets & Dashboard

Method	Endpoint	Description
`GET`	`/api/budgets`	List budgets for current month
`GET`	`/api/budgets/{year}/{month}`	List budgets for a specific month
`POST`	`/api/budgets`	Set a budget for a category

Method	Endpoint	Description
`PUT/DELETE`	`/api/budgets/{id}`	Update or remove a budget
`GET`	`/api/dashboard/summary`	Total income, expense, net for a period
`GET`	`/api/dashboard/by-category`	Spending breakdown by category
`GET`	`/api/dashboard/trends`	Monthly spending trends
`GET`	`/api/dashboard/top-merchants`	Top spending merchants

14. Security Design

Concern	Solution
Authentication	JWT with short-lived access tokens (15 min) and longer refresh tokens (7 days). Signed with server-side secret (HS256).
Password Storage	BCrypt with work factor 12. Never stored in plaintext.
Authorization	Every endpoint checks `userId` from JWT. Users can only access their own data.
CSRF	Not applicable — JWT sent in `Authorization` header (not cookies).
CORS	Whitelist frontend origin only. No wildcard `*`.
File Upload	Max 10 MB. Only `.csv` accepted. Content-type validated. Parsed in memory, never written to disk.
SQL Injection	Parameterized queries via JPA. No string concatenation in queries.
Rate Limiting	5 attempts/minute on auth endpoints to prevent brute-force attacks.
Data at Rest	PostgreSQL with encrypted volumes. Application-level encryption is a V2 consideration.

15. Development Roadmap

Phase 1 — Foundation (Weeks 1–2)

- Project scaffolding: Spring Boot + React + PostgreSQL + Docker Compose
- User registration and login (JWT authentication flow)
- Database schema + Flyway migrations for all 7 tables
- Basic category CRUD with seeded defaults (Food, Transport, Shopping, Bills, Entertainment, Health, Income, Transfer, Other)

Phase 2 — The Import Pipeline (Weeks 3–4)

- Bank Profile CRUD (column mapping, saved formats)
- CSV upload endpoint with Apache Commons CSV parsing
- SHA-256 fingerprint generation and deduplication logic
- Categorization rule engine with priority ordering
- Preview API endpoint (returns categorized + deduplicated results)
- Confirm API endpoint with batch tracking
- Import undo (batch delete by import_batch_id)

Phase 3 — Frontend Core (Weeks 5–6)

- Login / Register pages
- CSV upload wizard (file select → column mapping → preview → confirm)
- Transaction list with pagination, filtering, and inline editing
- Category management UI
- Rule management UI (create, edit, reorder, test)
- Import history page with one-click undo

Phase 4 — Budgets & Dashboard (Weeks 7–8)

- Budget CRUD API and UI with progress bars
- Dashboard API with SQL aggregation queries
- Spending by category chart (donut/pie)
- Income vs. expense chart (bar)
- Spending trends chart (line — last 6/12 months)
- Top merchants ranked list

Phase 5 — Polish & Production (Weeks 9–10)

- Email notifications for budget alerts (`@Scheduled` cron job)
- Responsive design (mobile-friendly layouts)
- Error handling with user-friendly messages
- Loading states, skeleton screens, empty states
- Unit tests (JUnit 5 + Mockito) and integration tests (Testcontainers)
- Frontend tests (Vitest + React Testing Library)
- Docker Compose for one-command deployment
- README with setup instructions, screenshots, and demo video

Stretch Goals (V2)

- Multi-account support (separate ledgers for different bank accounts)
- Transaction splitting (one purchase → multiple categories)
- Recurring transaction detection
- Export to CSV/PDF
- Dark mode and PWA support
- Shared household access (multiple users, one ledger)

16. Why This is a Strong Portfolio Project

For Interviewers

An interviewer can understand Ledgerly in 30 seconds: *"You upload a bank statement CSV, it categorizes your spending, and shows you dashboards."* No complex domain knowledge required. That instant comprehension is gold — it means the interviewer spends their time asking about your engineering, not trying to understand what the app does.

Technical Depth

Skill	How Ledgerly Demonstrates It
File parsing & data pipelines	The import pipeline handles messy real-world data: locale-aware number parsing, date format detection, edge cases like mid-file headers.
Deduplication	SHA-256 fingerprinting with database-backed uniqueness checks. Handling edge cases like same amount on the same day at the same place.
Rule engine	A configurable, priority-ordered matching system. Extensible to new operators without modifying existing code (Strategy pattern).
Preview/commit pattern	A two-phase write that mirrors real-world patterns in fintech and e-commerce (shopping cart → checkout). Strong talking point about data integrity.
SQL aggregation	Dashboard queries use `GROUP BY`, `SUM`, `CASE WHEN`, date functions, and window functions. Not just CRUD.
JWT security	Token-based auth with refresh flow, rate limiting. Shows understanding beyond "add Spring Security dependency."
Scheduled jobs	Budget alert emails run on a cron schedule. Demonstrates async processing and email delivery.

Skill	How Ledgerly Demonstrates It
Full-stack ownership	API, database, frontend, deployment config, and tests. End-to-end.

Conversation Starters in an Interview

1. *"Walk me through what happens when a user uploads a CSV."* → Explain the 7-stage pipeline with deduplication, rule matching, and atomic commits.
2. *"How do you handle duplicate transactions?"* → Explain fingerprinting, why SHA-256, and edge cases (same amount on same day).
3. *"Why preview before save?"* → Talk about data integrity, user trust, and how this pattern appears in production fintech systems.
4. *"How would you scale this?"* → Discuss batch processing, database indexing strategy, and eventual service extraction.

> **Presentation Tip:** When presenting this project, always lead with the *problem* ("banks give you raw data, budgeting apps want your credentials") and the *design decisions* ("preview before save, hash-based dedup, deterministic rules over ML") — not just the tech stack. That's what separates a portfolio project from a tutorial follow-along.

Built with Spring Boot, React, and PostgreSQL.

Ledgerly — Clarity from a CSV.